

UNIVERSIDAD DEL VALLE DE GUATEMALA
Facultad de Ingeniería

Docker, Docker Compose y microservicios

Diego Linares
Andy Fuentes
Christian Echeverria
Diederich Solis

Machine Learning Engineering

Agosto 18 de 2026

Introducción: el problema que motiva todo

En la Actividad 3 calibramos los hiperparámetros de un pipeline de scikit-learn sobre el dataset de la UEFA Champions League. Al ejecutarlo en dos computadoras distintas nos encontramos con algo revelador: **el mismo código, con la misma semilla `random_state=42`, dio puntajes distintos**. En Windows, GridSearchCV reportó un `f1_macro` de 0.563; en macOS, 0.611. La causa no era el azar sino que cada máquina tenía una versión distinta de scikit-learn instalada.

Ese hallazgo es exactamente el problema que Docker resuelve: **empaquetar la aplicación junto con su sistema operativo, sus librerías y sus versiones exactas**, de modo que se ejecute de forma idéntica en cualquier máquina. Fijar la semilla no alcanza para reproducir un experimento; hay que fijar también el entorno completo.

Docker

Contenedores frente a máquinas virtuales

Un contenedor **no** es una máquina virtual pequeña. Es un proceso normal del sistema anfitrión, aislado mediante funciones del kernel de Linux: los *namespaces* hacen que el proceso vea su propio sistema de archivos, su propia red y su propia tabla de procesos, mientras que los *cgroups* limitan cuánta CPU y memoria puede consumir.

	Máquina virtual	Contenedor
Qué virtualiza	El hardware completo	Solo el proceso
Sistema operativo	Cada VM lleva el suyo entero	Comparten el kernel del anfitrión
Tamaño típico	Gigabytes	Decenas o cientos de MB
Tiempo de arranque	Minutos	Segundos
Aislamiento	Total	A nivel de proceso

La consecuencia práctica es que en una laptop se pueden correr decenas de contenedores, pero apenas dos o tres máquinas virtuales.

Dockerfile, imagen y contenedor

Son tres conceptos que conviene no confundir:

Concepto	Qué es	Analogía
Dockerfile	La receta: instrucciones para construir	El plano de una casa
Imagen	El resultado, inmutable y de solo lectura	La casa construida
Contenedor	Una imagen en ejecución	La casa habitada

De una misma imagen se pueden levantar muchos contenedores. La imagen **nunca cambia**: lo que un contenedor escribe va a una capa temporal por encima, que **se elimina junto con el contenedor**. De esa propiedad se desprende toda la discusión de la sección *Volúmenes y persistencia de datos*.

Capas y caché de construcción

Cada instrucción del Dockerfile genera una **capa**, y Docker las almacena en caché: si una capa no cambió respecto a la construcción anterior, se reutiliza. Por eso en nuestros Dockerfiles las dependencias se copian e instalan **antes** que el código:

```
COPY entrenador/requirements.txt ./requirements.txt    # cambia poco -> capa cacheada
RUN pip install --no-cache-dir -r requirements.txt      # la instruccion lenta
COPY entrenador/entrenar.py ./                        # cambia mucho -> capa barata
```

Con el orden invertido, modificar una sola línea del script obligaría a reinstalar scikit-learn completo en cada construcción.

Comandos fundamentales

```
docker build -t mi-imagen .          # construir una imagen desde el Dockerfile
docker images                        # listar imagenes
docker run mi-imagen                 # levantar un contenedor
docker ps                            # contenedores corriendo (-a incluye detenidos)
docker logs <contenedor>             # ver su salida
docker exec -it <contenedor> sh      # abrir una terminal adentro
docker rm / docker rmi               # eliminar contenedor / imagen
```

Docker Compose

Docker administra un contenedor a la vez. Nuestra aplicación tiene tres, con dependencias reales entre ellos: la API no sirve de nada si el modelo todavía no existe, y no puede guardar nada si la base de datos aún no arrancó.

Docker Compose describe **el sistema completo en un archivo YAML** y lo levanta en el orden correcto con un solo comando:

```
docker compose up --build    # construir y levantar todo
docker compose ps            # estado de cada servicio
docker compose logs -f api   # seguir los logs de un servicio
docker compose down          # apagar y borrar contenedores (los volúmenes SE QUEDAN)
docker compose down -v       # apagar y borrar TAMBIEN los volúmenes (destrutivo)
```

Además de la comodidad, Compose aporta cuatro cosas que tendríamos que resolver a mano:

1. **Red privada automática.** Compose crea una red y los servicios se localizan por *nombre*: la API se conecta a `postgresql://...@db:5432/...`, literalmente `db`, sin necesidad de direcciones IP. Docker ejecuta un DNS interno.
2. **Orden de arranque real.** Mediante `depends_on` con condiciones: `service_completed_successfully` hace que la API espere a que el entrenador *termine correctamente*, y `service_healthy` la hace esperar a que Postgres pase su `healthcheck`. Esto sustituye al típico “esperar cinco segundos y confiar”.
3. **Configuración por variables de entorno**, leídas de un archivo `.env` que no se versiona en el repositorio.

4. **Aislamiento hacia el exterior.** Solo la API publica un puerto; Postgres es inalcanzable desde fuera aunque los contenedores sí lo vean entre ellos.

Adaptación del pipeline a microservicios

Por qué separar

En la Actividad 3 todo era un único programa que leía el CSV, calibraba, evaluaba e imprimía resultados. Eso funciona en una laptop, pero no en producción: entrenar tarda minutos y consume CPU, mientras que responder una predicción debe tardar milisegundos. Si ambas tareas comparten proceso, cada reentrenamiento deja la aplicación sin responder.

Los tres servicios

Servicio	Tipo	Qué hace	Ciclo de vida
entrenador	Por lotes	Lee el CSV, calibra con <code>RandomizedSearchCV</code> y guarda el modelo	Corre y termina
api	Servicio	Carga el modelo del volumen y responde predicciones por HTTP (FastAPI)	Siempre encendido
db	Estado	Guarda el historial de predicciones (PostgreSQL)	Siempre encendido

El flujo es el siguiente: el **entrenador** lee el dataset desde un *bind mount* de solo lectura, calibra los hiperparámetros y escribe `modelo.joblib` en un volumen compartido; luego termina y su contenedor muere. La **api** monta ese mismo volumen en modo solo lectura, carga el modelo y expone los endpoints `/salud`, `/modelo`, `/predecir` y `/predicciones`. Cada predicción se registra en **db**, que persiste en su propio volumen.

Ventajas concretas de esta separación

- **Escalado independiente:** ante mucha carga, `docker compose up --scale api=3` levanta tres copias de la API sin tocar el entrenador.
- **Reentrenar no interrumpe el servicio:** la API sigue sirviendo el modelo anterior mientras el entrenador produce el nuevo.
- **Imágenes más pequeñas y específicas:** la API no necesita el dataset ni el código de calibración.
- **Fallos aislados:** si Postgres se cae, la API continúa prediciendo y únicamente deja de registrar el historial.

El detalle técnico que hace posible todo esto

El modelo se serializa con `joblib`, y un objeto serializado **guarda la ruta de importación de sus clases, no su código**. Nuestro pipeline contiene los transformadores

PorcentajeATexto y RatioATexto, que son clases propias. Si la API no pudiera importarlas desde el mismo módulo, la carga del modelo fallaría.

Por ese motivo **ambas imágenes instalan el mismo paquete** (act3_pipeline-0.1.0-py3-none-any que empaquetamos en la Actividad 3, y ambos archivos `requirements.txt` fijan **exactamente las mismas versiones** de scikit-learn, pandas, numpy y scipy. Aquí se ve para qué servía el trabajo de empaquetado de la actividad anterior.

Volúmenes y persistencia de datos

El problema de fondo

El sistema de archivos de un contenedor es la imagen (de solo lectura) más una **capa escribible** superpuesta. Esa capa pertenece al contenedor: al eliminarlo, se elimina con él.

Si el entrenador guardara el modelo en `/app/modelo.joblib`, al terminar el contenedor el modelo desaparecería y la API nunca lo vería. Si Postgres escribiera su base en la capa escribible, cada `docker compose down` borraría el historial completo.

Las cuatro formas de manejar datos

Forma	Dónde vive	Sobrevive	Cuándo usarla
Capa escribible	Dentro del contenedor	No	Archivos temporales
Volumen con nombre	Área administrada por Docker	Sí	Datos de la aplicación: bases de datos, modelos
Bind mount	Una carpeta del anfitrión	Sí	Código, datos de entrada, configuración
tmpfs	Memoria RAM	Nunca toca disco	Secretos y temporales

En este proyecto usamos las dos intermedias:

volumes:

- `./datos:/datos:ro` # BIND MOUNT: carpeta del anfitrión, de solo lectura
- `modelos:/modelos` # VOLUMEN CON NOMBRE: lo administra Docker

Volumen con nombre frente a bind mount

	Volumen con nombre	Bind mount
Lo administra	Docker	El usuario
Ruta	Irrelevante para el usuario	Depende de cada máquina
Portabilidad	Alta	Baja
Rendimiento (Docker Desktop)	Mejor	Peor
Ideal para	Bases de datos, modelos	Código, datos de entrada

La regla práctica es simple: **si lo produce la aplicación, volumen con nombre; si lo escribe el desarrollador, bind mount.**

Por qué el modelo no va dentro de la imagen

Sería tentador entrenar durante la construcción de la imagen y dejar el modelo incorporado. Es una mala práctica por tres razones:

1. Reentrenar obligaría a reconstruir y redistribuir la imagen completa.
2. Las imágenes deben ser inmutables y reproducibles; entrenar durante la construcción las vuelve dependientes de los datos del momento.
3. El modelo es **estado**, no código, y se versiona de manera distinta.

Comandos de volúmenes

```
docker volume ls                # listar
docker volume inspect actividad4-mlops_modelos # ver donde vive realmente
docker volume rm <nombre>      # eliminar (destructivo)
docker volume prune             # eliminar los que nadie usa
```

Un problema real: permisos y volúmenes

Al levantar el sistema por primera vez, el entrenamiento se completó correctamente pero el guardado falló:

```
[entrenador] listo en 12.0 s | mejor f1_macro en CV = 0.619
[entrenador] modelo final: LogisticRegression
PermissionError: [Errno 13] Permission denied: '/modelos/modelo.joblib'
```

La causa es la interacción entre dos buenas prácticas que chocan entre sí. Por seguridad, el contenedor no corre como **root** sino como un usuario sin privilegios (**mlops**, uid 1000). Pero cuando Docker monta un volumen con nombre que está vacío, **copia el propietario y los permisos de esa carpeta tal como existe en la imagen**; si la carpeta no existe en la imagen, el volumen se crea perteneciendo a **root** y el usuario sin privilegios no puede escribir en él.

La solución es crear el directorio dentro de la imagen y asignarle el propietario correcto *antes* de que el volumen se monte encima:

```
RUN useradd --create-home --uid 1000 mlops \
    && mkdir -p /modelos \
    && chown -R mlops:mlops /app /modelos
USER mlops
```

Como el volumen ya se había creado con el propietario equivocado, hubo que eliminarlo con **docker compose down -v** para que naciera de nuevo con los permisos correctos. Es un error frecuente y vale la pena registrarlo: el volumen recuerda su propietario igual que recuerda sus datos.

Demostración de la persistencia

Las tres pruebas se ejecutaron sobre el sistema ya funcionando. Estos son los resultados reales.

Prueba 1: el modelo sobrevive a su creador.

SERVICE	STATE	STATUS
api	running	Up 15 seconds (healthy)
db	running	Up 32 seconds (healthy)
entrenador	exited	Exited (0) 15 seconds ago

```
$ curl http://localhost:8000/salud
```

```
{"estado": "ok", "contenedor": "72e91038c58f", "modelo_cargado": true, "base_de_datos": true}
```

El contenedor `act4-entrenador` ya terminó, y sin embargo la API tiene el modelo cargado y responde. Los metadatos indican que lo entrenó el contenedor `3e635acd60eb`, que ya no existe.

Con el modelo en servicio, dos predicciones sobre partidos del propio dataset:

Real Madrid vs Marseille (resultado real 2-1):

```
{"prediccion": "Home Win",  
  "probabilidades": {"Home Win": 0.8764, "Draw": 0.1185, "Away Win": 0.0051}}
```

Ajax vs Inter (resultado real 0-2):

```
{"prediccion": "Away Win",  
  "probabilidades": {"Away Win": 0.7822, "Draw": 0.1868, "Home Win": 0.0310}}
```

Prueba 2: la base sobrevive a un apagado completo. Tras `docker compose down`, `docker compose ps -a` no lista ningún contenedor, pero `docker volume ls` sigue mostrando `actividad4-mlops_modelos` y `actividad4-mlops_pgdata`. Al levantar de nuevo con `docker compose up -d`:

```
[entrenador] contenedor 8910543744d3 | python 3.12.14 | sklearn 1.8.0
```

```
[entrenador] ya existe /modelos/modelo.joblib (viene del volumen de una corrida anterior)
```

```
[entrenador] no se reentrena. Para forzarlo: FORZAR_REENTRENO=1
```

```
$ curl http://localhost:8000/predicciones
```

```
{"total_historico": 2, ...}
```

```
entrenado_en: 2026-08-19T02:44:37+00:00
```

```
contenedor que lo entreno: 3e635acd60eb
```

El entrenador es un contenedor *nuevo* (`8910543744d3`) que encontró el trabajo del anterior y decidió no repetirlo; las dos predicciones siguen en la base; y los metadatos del modelo siguen apuntando al contenedor original. Nada de eso sobreviviría sin volúmenes.

Prueba 3: sin volúmenes no queda nada. Con `docker compose down -v`:

```
Volume actividad4-mlops_modelos Removed
```

```
Volume actividad4-mlops_pgdata Removed
```

```
$ docker volume ls --filter name=actividad4
```

```
DRIVER    VOLUME NAME
```

```
(vacío)
```

Al levantar otra vez, el entrenador vuelve a trabajar desde cero y el historial se pierde:

```
[entrenador] calibrando con RandomizedSearchCV, 60 combinaciones, metrica f1_macro...  
[entrenador] listo en 14.9 s | mejor f1_macro en CV = 0.619  
[entrenador] modelo guardado en /modelos/modelo.joblib (10 KB)
```

```
$ curl http://localhost:8000/predicciones  
{ "total_historico": 0, "predicciones": [] }
```

Esa es exactamente la diferencia entre `down` y `down -v`, y la razón por la que la segunda se escribe con cuidado.

Comprobación de reproducibilidad

El resultado más importante de toda la actividad es este: el modelo entrenado **dentro del contenedor** dio exactamente los mismos números que en las dos computadoras de la Actividad 3.

	Windows (Act. 3)	macOS (Act. 3)	Contenedor (Act. 4)
Modelo ganador	LogisticRegression	LogisticRegression	LogisticRegression
f1_macro en CV	0.619	0.619	0.619
Accuracy en test	0.690	0.690	0.690
f1_macro en test	0.631	0.631	0.631

La diferencia respecto a la Actividad 3 es la garantía. Allá los números coincidieron por suerte: la discrepancia en los puntajes de las búsquedas basadas en Random Forest (0.563 contra 0.611) mostraba que las dos máquinas no estaban ejecutando lo mismo. Aquí la imagen fija scikit-learn 1.8.0, pandas 2.3.3 y Python 3.12.14, así que cualquier computadora que levante el contenedor ejecuta *exactamente* el mismo entorno. La causa del problema queda eliminada de raíz, no solo su síntoma.

Nota: el servicio entrenador ejecuta únicamente `RandomizedSearchCV`, que fue la búsqueda ganadora en la Actividad 3; no se volvieron a medir dentro del contenedor las otras dos búsquedas.

Evidencia de ejecución

Las capturas completas están en la carpeta `capturas/` del repositorio. Se incluyen aquí las cuatro más representativas.

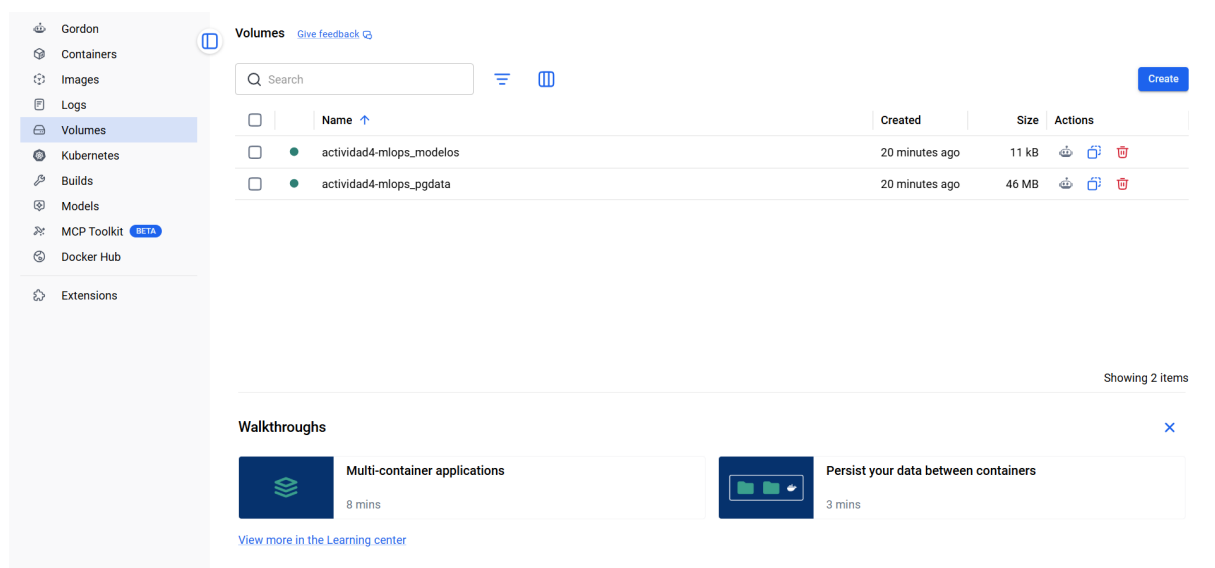


Figura 3: Los volúmenes con nombre administrados por Docker: `actividad4-mlops_modelos` guarda el modelo entrenado y `actividad4-mlops_pgdata` la base de datos. Existen de forma independiente de los contenedores.

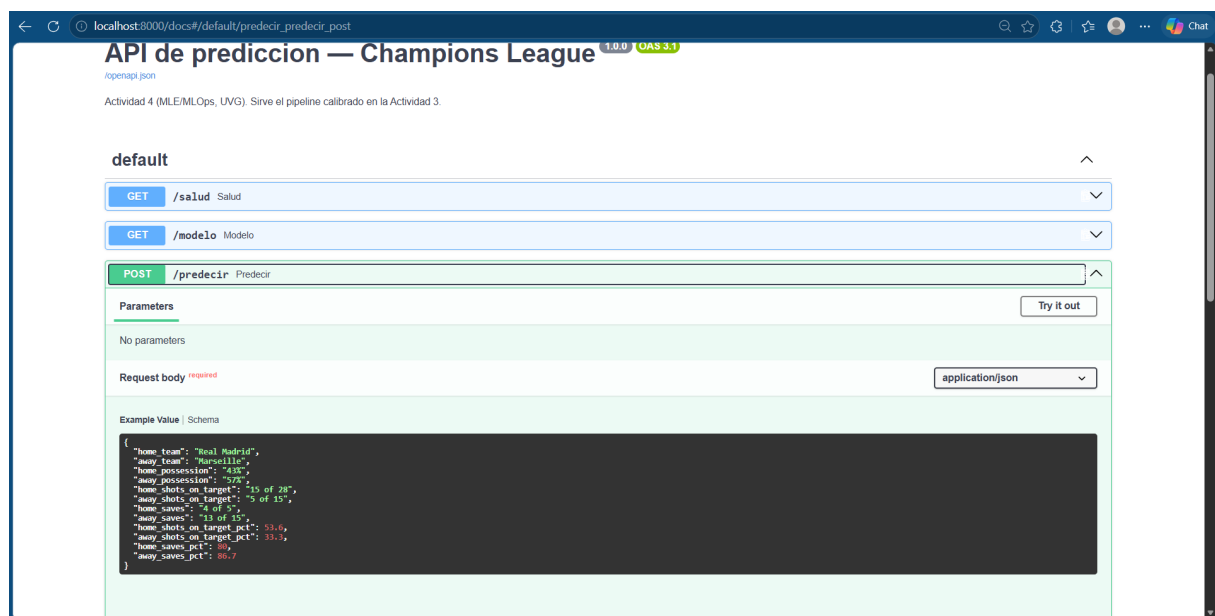


Figura 4: Documentación interactiva que FastAPI genera sola en `/docs`, con el endpoint `POST /predecir` desplegado. El cuerpo de ejemplo muestra el formato que espera el pipeline: la posesión como texto ("`43%`") y los tiros como "`15 of 28`", tal como vienen en el CSV original.

Conclusiones

1. **Docker resuelve un problema que ya habíamos sufrido.** La discrepancia de puntajes entre Windows y macOS en la Actividad 3 se debía a versiones distintas de scikit-learn. Fijar las versiones dentro de una imagen elimina esa clase completa de errores.
2. **El empaquetado de la Actividad 3 rindió aquí.** Ambas imágenes instalan el mismo wheel, y no por comodidad: sin él, la API no podría reconstruir el pipeline serializado porque no encontraría las clases de los transformadores propios.
3. **Separar el entrenamiento del servicio es la decisión de arquitectura central,** porque ambas tareas tienen ritmos y requisitos opuestos: una es pesada y esporádica, la otra liviana y constante.
4. **Sin volúmenes, un contenedor no recuerda nada.** El modelo y la base viven fuera del ciclo de vida de los contenedores; los datos de entrada entran por bind mount. Cada tipo de dato tiene su mecanismo y confundirlos se paga con pérdida de información.
5. **Las condiciones de `depends_on` sustituyen las esperas arbitrarias:** la API arranca cuando el entrenador terminó correctamente y Postgres responde, no cuando transcurrió un tiempo fijo.
6. **Correr sin privilegios exige preparar los volúmenes.** El `PermissionError` que nos apareció no fue un descuido aislado: es la consecuencia predecible de montar un volumen vacío en un contenedor que no corre como `root`. Un volumen recuerda su propietario igual que recuerda sus datos.

Limitaciones y trabajo futuro

El entrenador se ejecuta una única vez al levantar el sistema; en producción sería una tarea programada o disparada por la llegada de datos nuevos. La API carga el modelo al arrancar, de modo que un reentrenamiento exige reiniciarla. El modelo se guarda con un único nombre, sin versionado: un registro de modelos (por ejemplo MLflow) sería el siguiente paso natural. La contraseña de la base viaja como variable de entorno, cuando lo correcto en producción sería usar *secrets*. Finalmente, sigue vigente la limitación del modelo señalada en la Actividad 3: utiliza estadísticas que solo se conocen cuando el partido ya terminó, de manera que explica el resultado más de lo que lo predice.